Project – Low Level Design

on

Non-Profit Support Triage Agent

Course Name: AgenticAI

Institution Name: Medicaps University – Datagami Skill Based Course

Submitted by:

Sr no	Student Name	Enrolment Number
1.	Ananya Subramanya Rao	EN22CS301120
2.	Anirudh Vyas	EN22CS301131
3.	Anushka Kashyap	EN22CS301180
4.	Atharv Chaturvedi	EN22CS301228
5.	Harshit Panchal	EN23CS3L1010

Group Name: Group 03D2

Project Number: AAI-15

Industry Mentor Name: Ashwin Krishnan Sir

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: 2026

Table of Contents

1. **Introduction**
   1.1. Scope of the document
   1.2. Intended Audience
   1.3. System overview
2. **System Design**
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. **References**

1. Introduction

In today's fast-paced digital world, non-profit organizations face a significant challenge in managing large volumes of incoming messages. Manually triaging each message is time-consuming, error-prone, and leads to delayed responses and overburdened staff.

TriageAgent is a production-grade, AI-powered communication triage system designed to solve this problem. It works like a smart AI receptionist for a non-profit. It leverages the power of Large Language Models and NLP to automate the initial response workflow for non-profit communications. The system processes every incoming message in real time, classifying it by urgency and intent, extracting critical data points, generating a professional draft response, and hands it to the right person for final approval.

Built on a modern full-stack architecture using **FastAPI** on the backend and **React** on the frontend, TriageAgent integrates **Google Gemini API** as its core LLM, **spaCy** for fast offline NER, **LangChain** for multi-agent orchestration, and **Firebase** for authentication and real-time database storage. The system follows a multi-agent AI architecture where four specialized agents - Classifier, NER, Response Generator, and Router — work together under a central Orchestrator to deliver accurate and professional results.

Problem Statement

Create a production-grade reactive triage agent for Non-Profit communications, utilizing Large Language Models to automate initial response workflows. The system performs real-time classification of incoming text to determine urgency and intent, applies Named Entity Recognition (NER) to extract critical data points such as IDs, dates, and generates contextually accurate draft responses using an LLM API to accelerate resolution times.

Project Objectives

- **Real-Time Classification**: Classify every message by urgency (Critical /High /Normal/Low) and intent (Donation/Volunteer/Complaint/etc.) using Gemini API.
- **Named Entity Recognition**: Extract names, dates, amounts, emails, receipt IDs, and case numbers using hybrid spaCy + LLM approach.
- **Draft Response Generation**: Generate professional, contextual email responses tailored to message content and urgency.
- **Intelligent Routing** — Auto-route messages to the correct department.
- **Human-in-the-Loop**: Admin dashboard to review, approve, or reject AI responses.
- **Analytics**: Real-time insights on urgency trends, intent distribution, and system performance.
- **Production-Grade Reliability**: Secure, scalable system with JWT auth, rate limiting, retry logic, and Firebase infrastructure.

1.1 Scope of the document

This Low-Level Design (LLD) document provides a detailed technical specification for the **TriageAgent** system — An AI-Powered Communication System for Non-Profit Organizations. It translates the high-level architecture into concrete, implementable components, covering module design, API contracts, data models, class descriptions, user flow, data flow, and key design decisions. The document covers:

- Overall system architecture and component interactions
- Technology stack and design decisions
- Data flow between frontend, backend, and third-party services
- Integration design for AI (Google Gemini), AI Agents, and Firebase
- Security, authentication, and API design at a high level

This document serves as the primary technical reference for developers, reviewers, and evaluators of the project. It covers both the backend (FastAPI + AI agents) and the frontend (React 18) subsystems.

1.2 Intended Audience

1. **Non-Profit Organization Staff** The primary users. Staff members who receive and respond to incoming communications daily — donation queries, volunteer inquiries, complaints, and partnership requests. They use the admin dashboard to review AI-generated classifications and approve or reject draft responses.
2. **Non-Profit Administrators / Managers** Team leads and managers who monitor the analytics dashboard to track message volume, urgency distribution, response times, and team performance metrics.
3. **Department Teams** Specific teams (Donations, Volunteers, Support, Partnerships) who receive routed messages assigned to them based on intent classification.

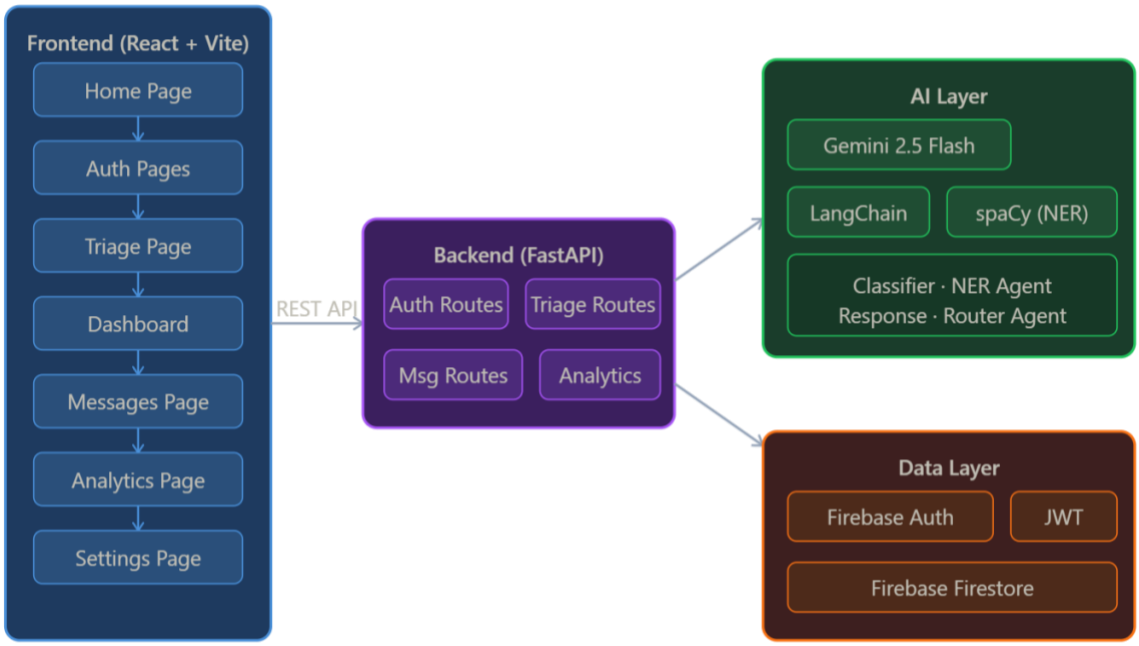
1.3 System overview

TriageAgent is a production-grade, AI-powered reactive triage system built for Non-Profit organizations. It automates the processing of incoming communications by performing four core operations in sequence:

#	Operation	Technology Used	Output
1	Message Classification	Gemini 2.5 Flash (JSON mode, temp=0.3)	urgency + intent + confidence + reasoning
2	Named Entity Recognition	spaCy en_core_web_sm + Gemini LLM (hybrid)	persons, dates, amounts, emails, receipt IDs
3	Draft Response Generation	Gemini 2.5 Flash (temp=0.7, max_tokens=2000)	Full email draft with Subject line + body
4	Intelligent Routing	Router Agent (enum lookups)	Department + priority level 1–4

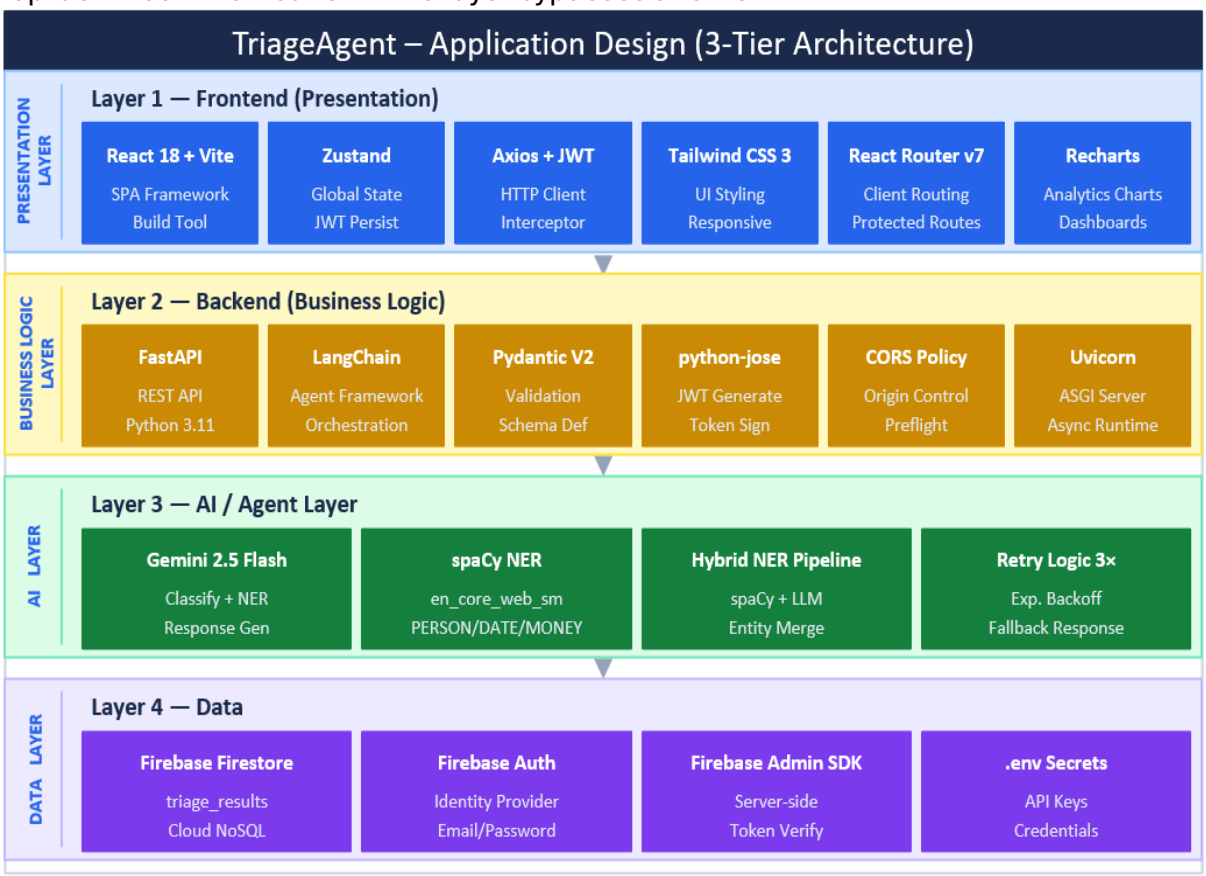
The system processes each message in 5–10 seconds, reducing manual triage time from 15–20 minutes to under 1 minute — a 10–20× improvement in resolution speed. All results are stored in Firebase Firestore and reviewed via a React 18 admin dashboard.

2. System Design

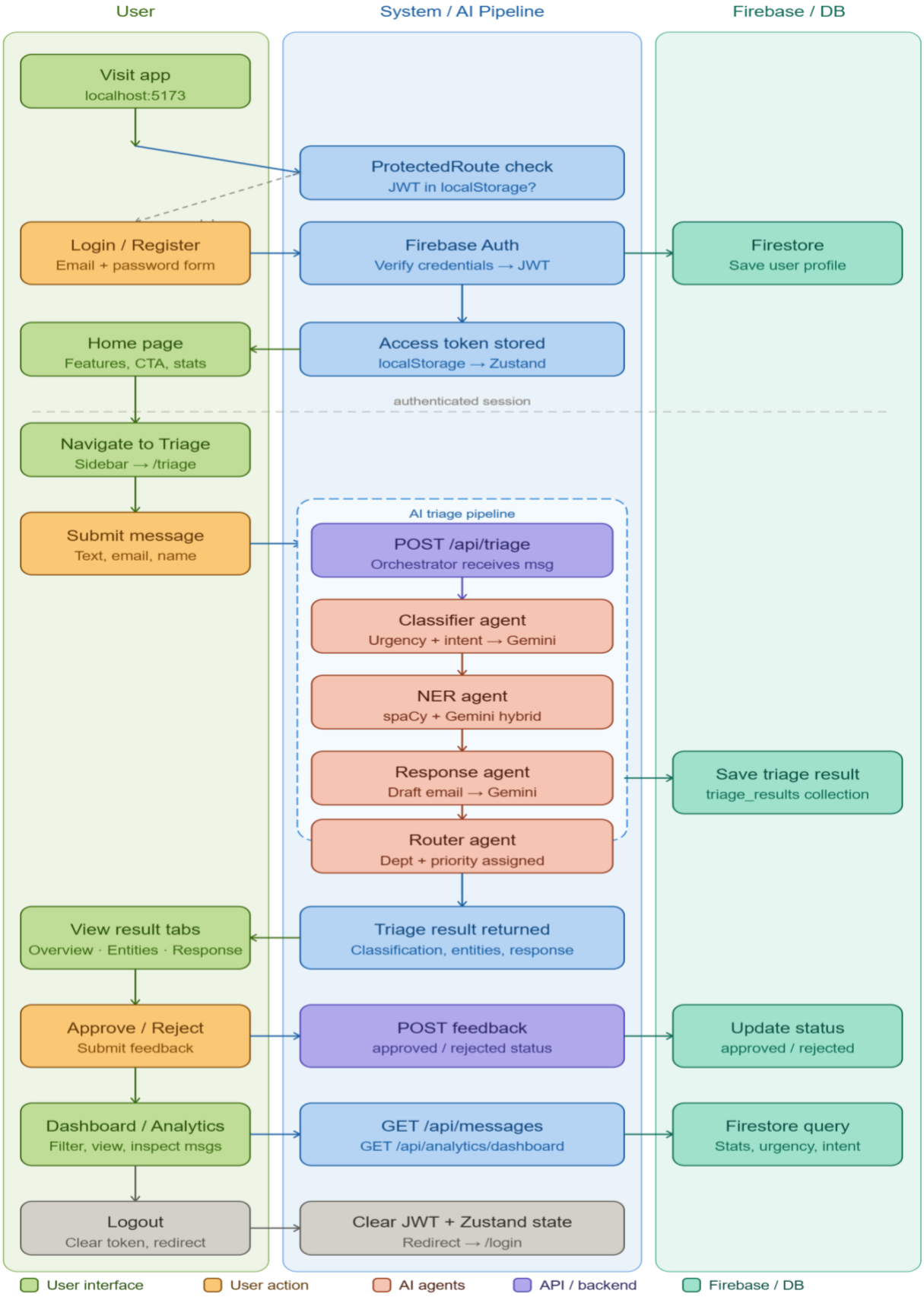


2.1 Application Design

The TriageAgent follows a 3-Tier Client-Server architecture with a Multi-Agent AI layer embedded in the backend tier. The system is organized into four distinct layers with strict top-down communication — no layer bypasses another.

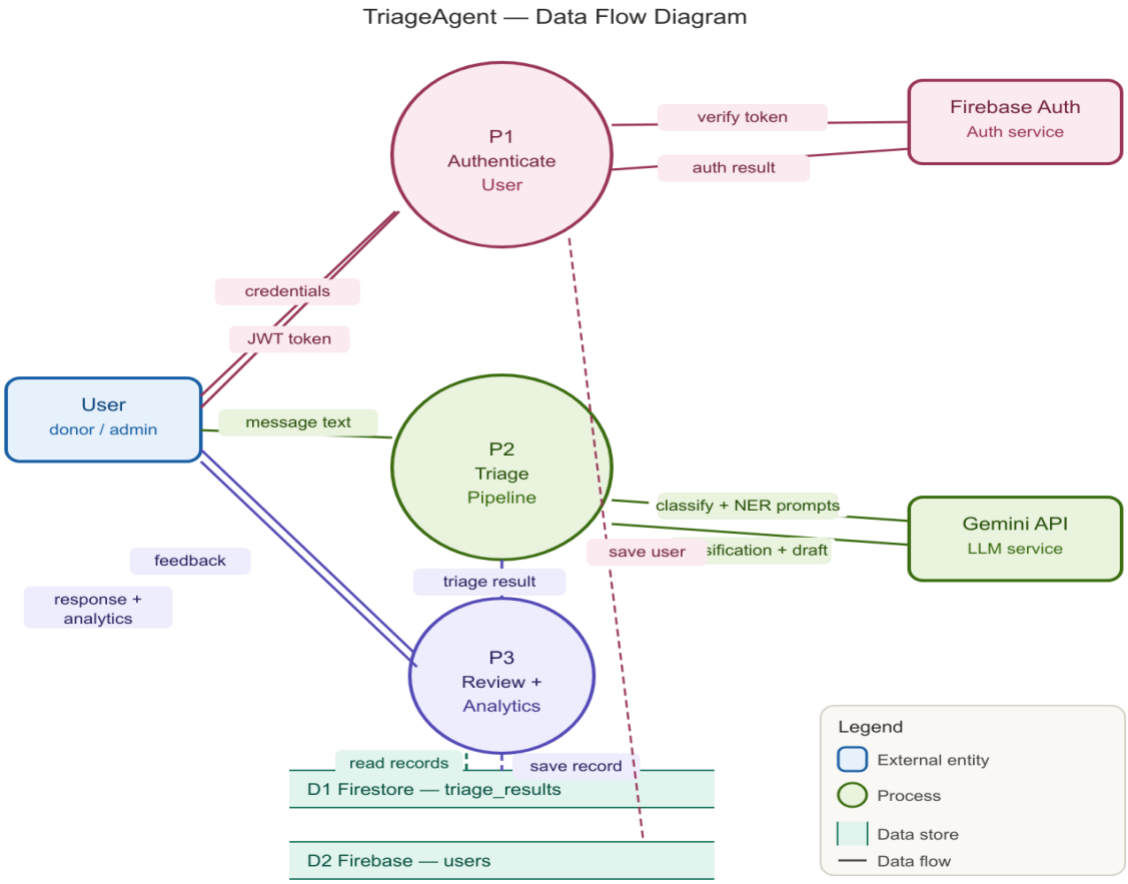


2.2 Process Flow

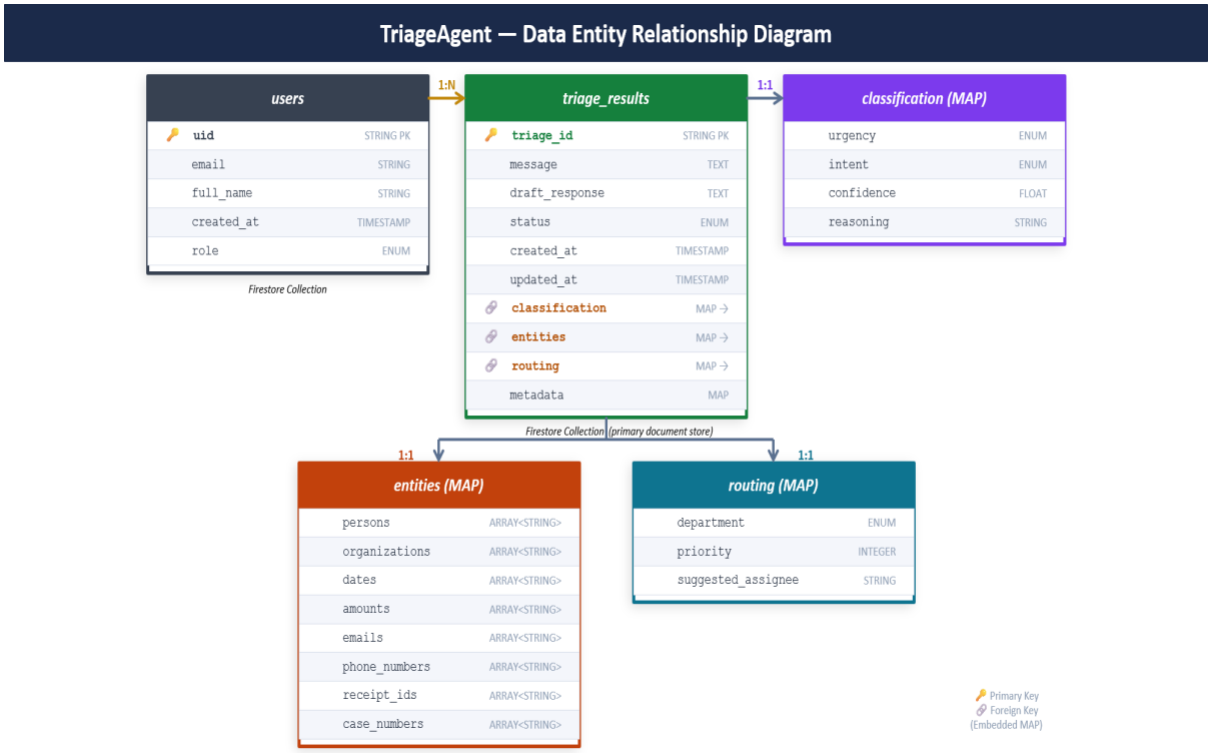


2.3 Information Flow

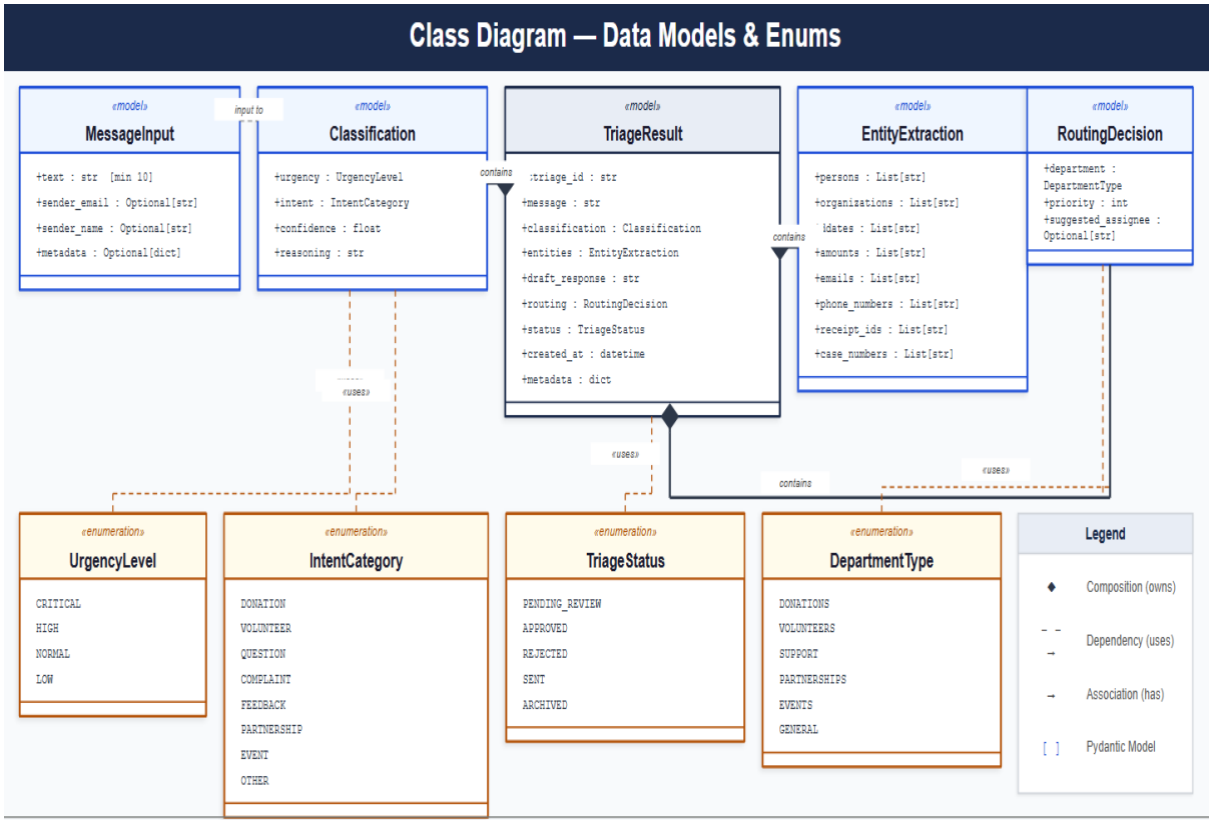
2.3.1 Data Flow Diagram

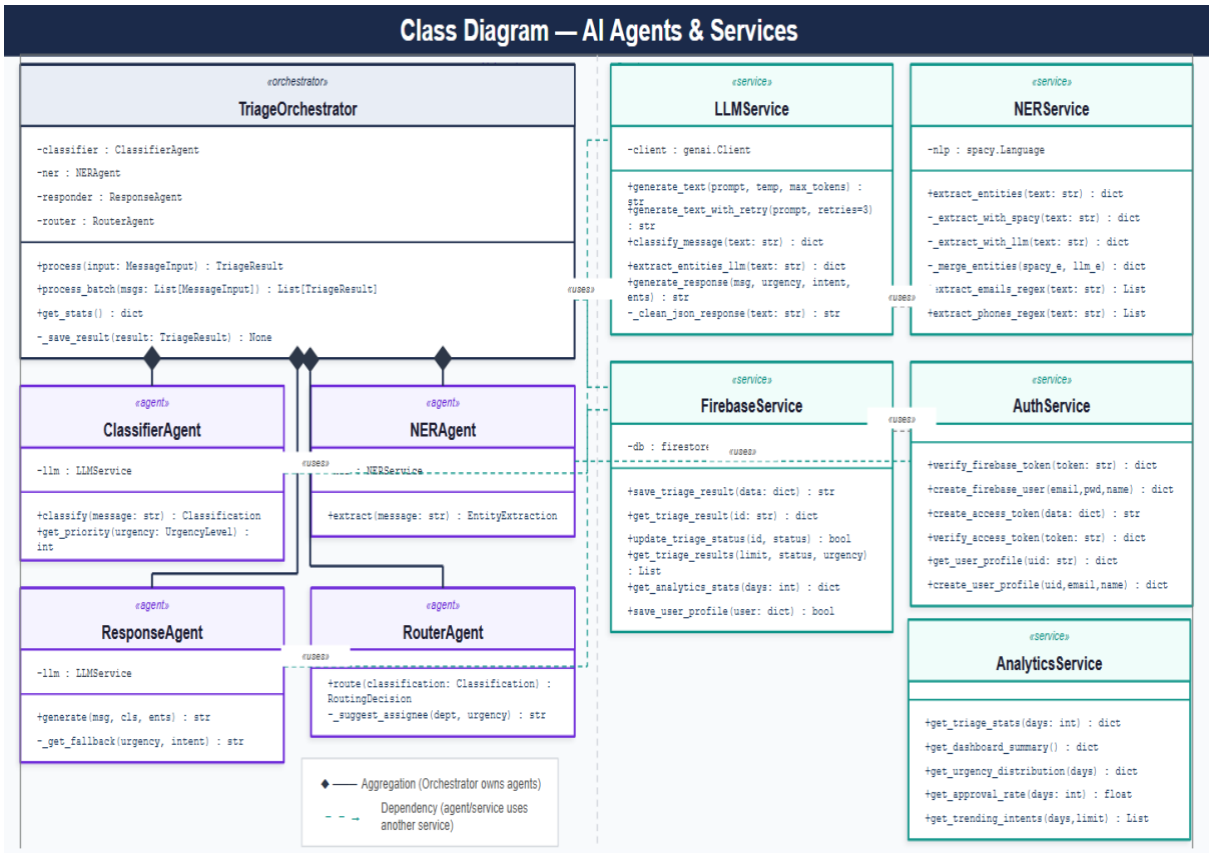


2.3.2 Data Entity Relationship Diagram



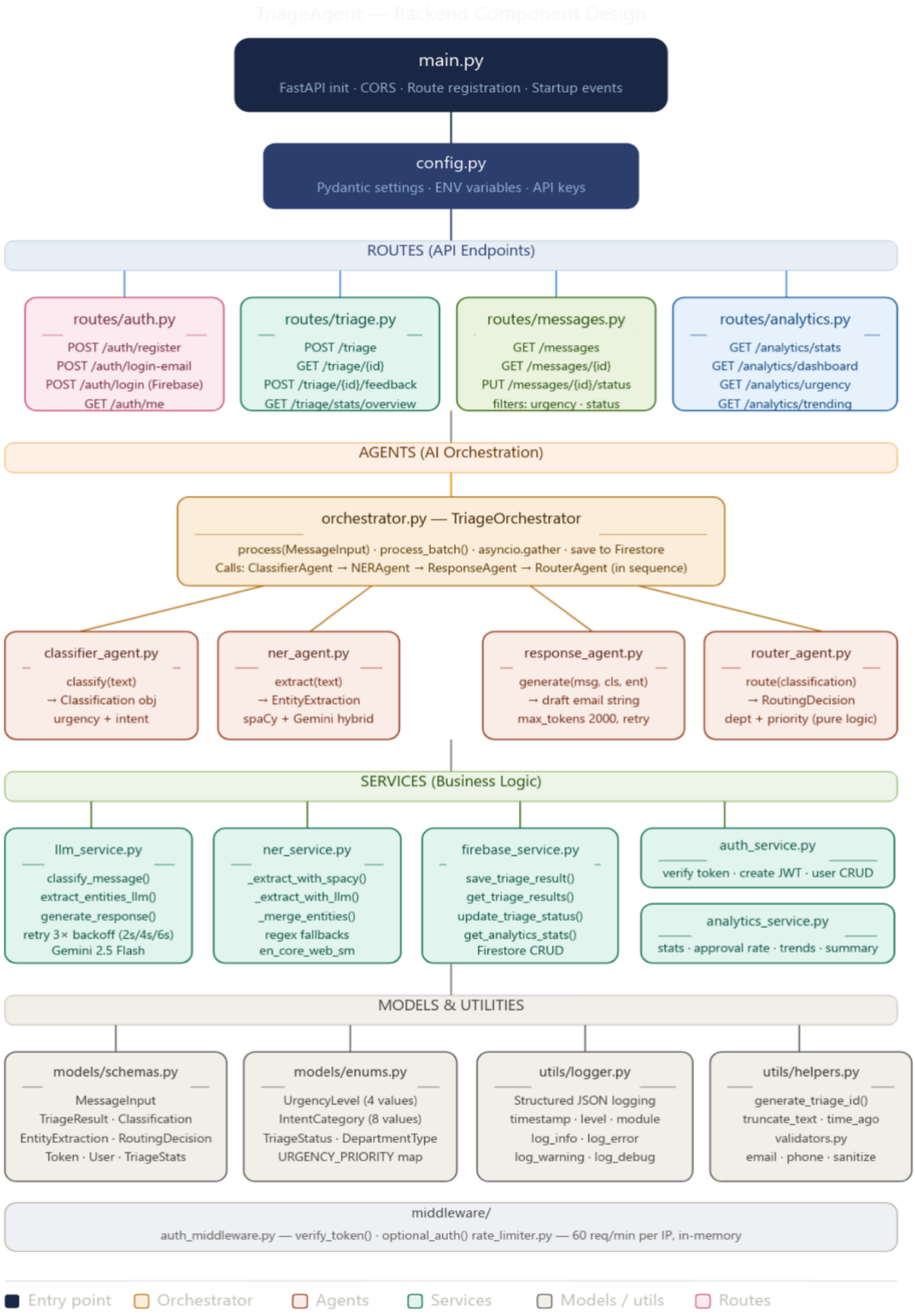
2.3.3 Class Diagram



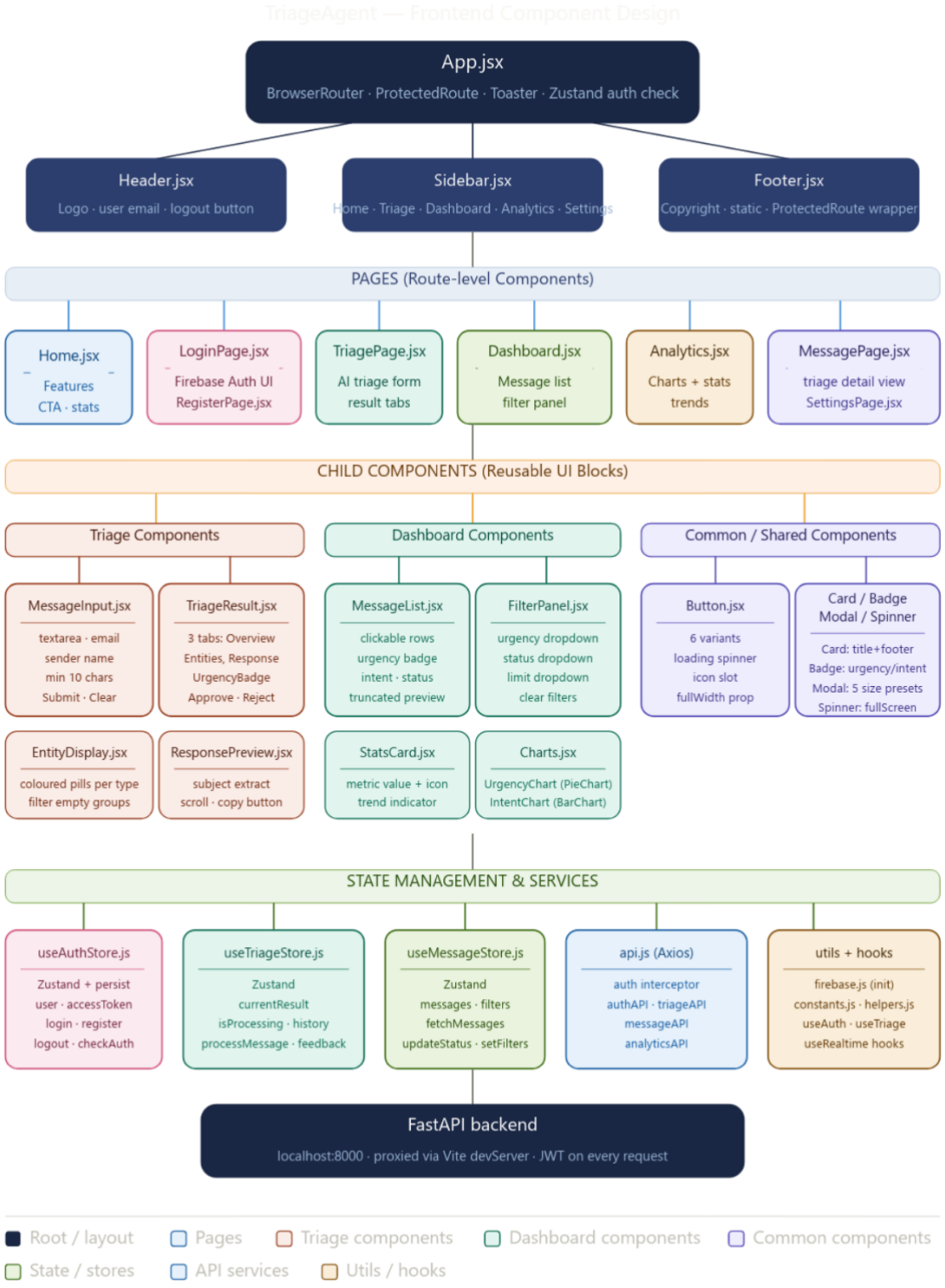


2.4 Components Design

2.4.1 Backend Components Design



2.4.2 Frontend Components Design



2.5 Key Design Considerations

No.	Design Consideration	Description	Why This Choice Was Made
1	**Hybrid NER Approach**	Combined two methods for entity extraction: **spaCy** for fast offline extraction (names, dates) and **Gemini LLM** for context-aware extraction (receipt IDs, emails, case numbers).	spaCy alone misses domain-specific entities, while LLM-only solutions are slower and costlier. A hybrid approach provides both **speed and accuracy**.
2	**LangChain over CrewAI**	Used **LangChain** as the framework to build and manage the agent pipeline.	LangChain provides strong **tool integration, prompt management, and pipeline control**, making it easier to orchestrate tasks and integrate LLM workflows compared to CrewAI for this project.
3	**Multi-Agent Architecture**	The system is divided into **four specialized agents**: Classifier Agent, NER Agent, Response Agent, and Router Agent coordinated by an **Orchestrator**.	Follows the **Single Responsibility Principle**. Each agent performs one specific task, making the system easier to **test, debug, maintain, and scale**.
4	**Retry Logic for LLM Calls**	Implemented retry logic for **Gemini API calls**, retrying up to **3 times with exponential backoff** (2s → 4s → 6s).	Gemini APIs sometimes return **503 errors (overloaded)**. Retry logic ensures the system **recovers automatically instead of failing**.
5	**Human-in-the-Loop**	AI generates the response, but **a human must approve it before sending**.	LLMs may produce **hallucinated or incorrect responses**. Human review ensures **accuracy, empathy, and reliability**, especially important for non-profit communication.
6	**Firebase over SQL Database**	Used **Firebase Firestore (NoSQL)** instead of PostgreSQL or SQLite.	Firestore requires **no server management**, offers **automatic scaling**, has a **free tier suitable for MVP**, supports **real-time updates**, and integrates easily with **Firebase Authentication**.
7	**JWT + Firebase Auth Dual Token System**	Authentication handled by **Firebase Auth**, while **JWT tokens** are used for backend API access.	Firebase securely manages **user credentials**, while JWT allows **stateless API authentication**, avoiding repeated Firebase verification calls and improving performance.

2.6 API Catalogue

Base URL: http://localhost:8000 | Auth: Authorization: Bearer {JWT} | Content-Type: application/json

Authentication APIs — /api/auth

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register new user, create Firebase account + Firestore profile	No
POST	/api/auth/login	Login with email and password, Verify Firebase ID token, return JWT	No
GET	/api/auth/me	Get current authenticated user profile	Yes

Triage APIs — /api/triage

Method	Endpoint	Description	Auth Required
POST	/api/triage	Process message through full AI triage pipeline	Yes
GET	/api/triage/{triage_id}	Get single triage result by ID	Yes
POST	/api/triage/{triage_id} / feedback	Submit human approval or rejection with optional edited response	Yes

Messages APIs — /api/messages

Method	Endpoint	Description	Auth Required
GET	/api/messages	Get all triage messages with optional filters (urgency, status, limit)	Yes
GET	/api/messages/ {triage_id}	Get full triage result document by ID	Yes
PUT	/api/messages/ {triage_id}/ status	Update message status (approved / rejected / sent / archived)	Yes

Analytics APIs — /api/analytics

Method	Endpoint	Description	Auth Required
GET	/api/analytics/stats	Get total message count and breakdown by urgency, intent, status	Yes
GET	/api/analytics/ dashboard	Get full dashboard summary — 7d + 30d stats, trends, performance	Yes
GET	/api/analytics/ urgency	Get message distribution by urgency level	Yes
GET	/api/analytics/intent	Get message distribution by intent category	Yes
GET	/api/analytics/ performance	Get avg/min/max processing times and approval rate	Yes

3. References

#	Reference	URL / Source
1	FastAPI Official Documentation	https://fastapi.tiangolo.com/
2	LangChain Documentation	https://python.langchain.com/docs/
3	Google Gemini API Documentation	https://ai.google.dev/gemini-api/docs
4	spaCy NLP Documentation	https://spacy.io/usage
5	Firebase Firestore Documentation	https://firebase.google.com/docs/firestore
6	Pydantic V2 Documentation	https://docs.pydantic.dev/
7	React 18 Documentation	https://react.dev/
8	Tailwind CSS Documentation	https://tailwindcss.com/docs
9	Zustand State Management	https://github.com/pmndrs/zustand
10	Project GitHub Repository	https://github.com/anushkakashyap31/TriageAgent

Technology Stack — TriageAgent

The following table specifies every technology used, its version, its role in the system, and the justification for choosing it over available alternatives.

Backend

#	Category	Technology	Version	Purpose	Justification
1	Backend Framework	FastAPI	0.109.0	REST API server — all routes, middleware, CORS, startup events	Fastest Python web framework (ASGI). Native Pydantic integration for request validation. Chosen over Flask/Django for async support and auto-generated API documentation.
2	Runtime	Python	3.11.0	Backend language for all agents, services, and API logic	Required by spaCy and LangChain ecosystem. Python 3.11 brings significant performance improve.

3	ASGI Server	Uvicorn	0.27.0	Runs the FastAPI app in development and production	Lightweight, production-ready ASGI server. Supports hot-reload in development.
4	Data Validation	Pydantic V2	2.5.3	Schema validation for all API request/response models	Pydantic V2 is 5–50x faster than V1. Strict type enforcement at API boundary.

AI / LLM

#	Category	Technology	Version	Purpose	Justification
5	LLM API	Google Gemini 2.5 Flash	google-genai 1.46	Message classification, entity extraction, draft response generation	Free tier available for MVP development. Fast inference speed suitable for real-time triage.
6	Agent Framework	LangChain	0.2.17	LLM chain management, prompt templates, output parsers, agent orchestration	More control over custom triage logic than CrewAI. Flexible for single-pipeline orchestration. Supports prompt templates with structured output parsing.

NLP / Named Entity Recognition

#	Category	Technology	Version	Purpose	Justification
7	NLP / NER	spaCy	3.7.2	Offline entity extraction — names, dates, organizations, money amounts	Runs offline with zero API cost — milliseconds per request. en_core_web_sm is accurate for standard English entities. Hybrid with Gemini gives best accuracy. spaCy alone misses domain-specific IDs; Gemini alone is slow and costly for every call.

Database

#	Category	Technology	Version	Purpose	Justification
8	Database	Firebase Firestore	firebase-admin 7.2	NoSQL document store for triage results, user	No server management required. Scales automatically on Google infrastructure. Free tier

				profiles, analytics stats	sufficient for MVP. Real-time listeners available for future live dashboard. Direct integration with Firebase Auth.

Authentication & Authorization

#	Category	Technology	Version	Purpose	Justification
9	Authentication	Firebase Auth	firebase-admin 7.2	User registration, login, password hashing, credential management	Handles password hashing and storage securely. Supports email/password. Integrated with Firestore same project credentials. Breach detection and security monitoring built-in.
10	API Authorization	JWT (python-jose)	3.3.0	Stateless API access token issued after Firebase login	Stateless backend — no session storage needed. JWT verified on every API request without hitting Firebase. Enables Postman / API testing without Firebase SDK.

Frontend

#	Category	Technology	Version	Purpose	Justification
11	UI Framework	React 18	19.2.0	Admin dashboard — triage interface, message review, analytics	Component-based architecture suits dashboard with reusable UI blocks. Virtual DOM ensures fast re-renders for real-time triage results. Huge ecosystem of libraries.
12	Build Tool	Vite	7.3.1	Dev server, hot module replacement, production bundler	Significantly faster than Create React App. Hot module replacement in milliseconds. Built-in proxy for /api calls eliminates CORS issues in development.

13	CSS Framework	Tailwind CSS	3.4.1	Utility-first styling for all components	Rapid UI development with utility classes. No context-switching between CSS files. Consistent design system. Used Tailwind v3 for PostCSS compatibility with Vite.

State Management & HTTP

#	Category	Technology	Version	Purpose	Justification
14	State Management	Zustand	5.0.11	Global state for auth, triage results, message list, filters	Minimal boilerplate compared to Redux. Three isolated stores (Auth, Triage, Message) prevent state coupling. Built-in persist middleware syncs accessToken to localStorage.